Chapter 3 Exercises

State width, signedness, rounding mode, and iteration number. Use binary32 for unspecified floating-point questions.

CH03-Q01 Addition and subtraction

Use eight-bit two's complement.

1. Find bits, signed value, and overflow for `0x7f + 0x01`.
2. Do the same for `0x80 + 0xff`.
3. Find bits and overflow for `0x80 - 0x01`.
4. Express signed overflow using carries at the top bit.

CH03-Q02 Sequential multiplication

Use four-bit unsigned shift-and-add with four-bit A and Q. Each addition temporarily retains carry C, so C:A=A+M. Shift the nine-bit `(C,A,Q)` right, moving C into A's MSB and discarding processed Q0. Initially A=0000,M=1101=13,Q=1011=11.

1. List old Q0, C:A, and A/Q after every shift.
2. Decode the final eight-bit product and check decimal multiplication.
3. Explain the general n-bit correctness using $\sum q_k(M2^k)$. A four-round trace alone is insufficient.
4. For n=32, add and shift each cost one cycle, setup three, and finalization two. Find serial latency. If an implementation accepts a new operation every four cycles, find throughput and state the extra hardware/context needed.

CH03-Q03 Restoring division

Use five-bit remainder A, four-bit quotient Q, divisor $M = 00101_2$, and initial dividend $Q = 1101_2$. Each iteration shifts combined state left, subtracts M, and either retains the nonnegative result with quotient bit 1 or restores it with bit 0.

1. Trace four rounds.
2. Find quotient and remainder.
3. Verify $D = QM + R$ and the remainder bound.
4. Analyze signed 32-bit `0x80000000 / 0xffffffff`.

Chapter 3 Exercises (continued)

CH03-Q04 Binary32 encoding

1. Encode −12.5 in hexadecimal.
2. Decode `0x3f000000`.
3. Give fields and powers of two for the smallest positive normal and subnormal.
4. Classify `0x7f800000` and `0x7fc00001`.

CH03-Q05 Nearest-even

Compute binary32 $1 + 2^{-24}$ using nearest-even.

1. Give the neighboring candidates.
2. Identify the tie and lowest retained bit.
3. State the result and status.
4. Explain how retaining greater intermediate precision can affect a later subtraction by 1.

CH03-Q06 Floating-point multiplication

Let binary32 $a = 1.5$ and $b = 2^{-126}$.

1. Give significands and unbiased exponents.
2. Find the exact significand product and exponent.
3. How many normalization left shifts are needed?
4. Why is adding stored exponent fields insufficient?

CH03-Q07 Nonassociativity

Use binary32 nearest-even with $x = 2^{24}, y = 1$.

1. Compute $(x + y) - x$ in order.
2. Compute $(x - x) + y$.
3. Locate the rounding that causes the difference.
4. Compare $(1 + 2^{-24}) - 1$ when an x87-style wider intermediate retains the sum exactly and when every operation rounds to binary32.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 3 Exercises (continued)

CH03-Q08 Arithmetic timing

A sequential multiplier takes 32 one-cycle iterations, three setup cycles, and two finalization cycles. A radix-4 model takes 16 iterations of 1.5 cycles each and six combined setup/finalization cycles. Both systems explicitly have enough replicated units or interleaved contexts to accept a new operation every eight cycles. A single unpipelined iterative unit does not naturally provide this capability.

1. Find each single-operation latency.
2. Find each steady-state throughput.
3. When does latency matter, and when does throughput matter?
4. Why do 16 iterations not imply twice the speed? What additional costs would a divider need?

CH03-Q09 Rounding and exceptions

Use binary32 round-to-nearest-even. Interpret values below the minimum positive subnormal by their exact mathematical value.

1. Give G/R/S for `1.0+2^-24`, its result, and inexact status.
2. Give G/R/S and result for `1.0+3*2^-25`.
3. Round 2^{-151} and 2^{-150}. For the latter, identify both candidates, the selected candidate, and flags.
4. Give default results and exceptions for maximum finite binary32 plus itself and `0*infinity`.

CH03-Q10 Single and separate rounding

Let binary32 $a = b = 1 + 2^{-23}$ and $c = -(1 + 2^{-22})$.

1. Give a and c in hexadecimal.
2. Execute multiply then add c, retaining the exact product, rounded product, and final bits.
3. Compute the exact result and bits of a fused RNE operation with one rounding.
4. Use $x = 2^{24}$ to compare `(x+1)-x` and `(x-x)+1`. Identify the changed rounding point and why exact associativity does not transfer to per-operation binary32 arithmetic.

Chapter 3 Exercises (continued)

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study